Universidad Centroamericana José Simeón Cañas
Departamento de Electrónica e Informática.

Administración de bases de datos
Laboratorio 4 Ejercicios G9

Grupo 9 - AdventureWorks - Finanzas

Base de datos a utilizar: AdventureWorks2022
Entorno: contenedor Docker `sqlserver-2025` de la clase (`localhost,1434`, login `sa`)

Indicaciones

Restaurar la base de datos **AdventureWorks** desde su archivo `AdventureWorks2022.bak` (el instructor indicará la ruta dentro del contenedor). Trabaja con el nombre `AdventureWorks2022`.

El laboratorio consta de 4 ejercicios prácticos. Debes elegir y resolver **3**. Los ejercicios son independientes entre sí.

Entrega un archivo `.sql` y el PDF `tu-carnet_Laboratorio4.pdf` con capturas. Activa siempre el plan de ejecución real (`Ctrl + M`) y `SET STATISTICS IO ON`.

En las tablas pequeñas el optimizador puede mantener un Scan aunque exista un índice. Si te ocurre, documenta las lecturas lógicas y explica la decisión del motor.

Trabaja siempre sobre una base recién restaurada y no modifiques los datos existentes, salvo donde el ejercicio lo pida.

Ejercicio 1 · Esquemas y secuencias

Problema. Tesorería quiere mover el catálogo de monedas a un esquema propio y llevar en él una bitácora de auditoría, cuyos identificadores los genere automáticamente una secuencia en lugar de escribirse a mano.

Requerimiento

1. Crear el esquema `Tesoreria` con propietario `dbo`.
2. Transferir la tabla existente `Sales.Currency` al nuevo esquema con `ALTER SCHEMA ... TRANSFER` y comprobar su nueva ubicación en el catálogo del sistema.
3. Crear la secuencia `Tesoreria.seq_bitacora_tesoreria` de tipo entero, que inicie en 1500, avance de 5 en 5, no haga ciclo y reserve 20 valores en memoria (CACHE).
4. Crear una **nueva tabla** `Tesoreria.Bitacora` con las columnas `id_registro` (INT), `usuario` (VARCHAR(50)) y `fecha` (DATE). Configura `id_registro` para que, por defecto (DEFAULT), tome el siguiente valor de `Tesoreria.seq_bitacora_tesoreria`.
5. Insertar dos filas en `Tesoreria.Bitacora` indicando únicamente el usuario y la fecha, y consultar la tabla para verificar que la secuencia asignó los identificadores correctamente.

Salida esperada. La tabla `Currency` aparece en el esquema `Tesoreria` y ya no en el original, y la nueva tabla `Bitacora` queda creada en ese mismo esquema. La consulta al catálogo de secuencias muestra `seq_bitacora_tesoreria` con valor inicial 1500 e incremento 5. Las dos filas insertadas reciben automáticamente los identificadores 1500 y 1505 (sin que los escribas en el `INSERT`), junto con el usuario y la fecha indicados.

Ejercicio 2 · Índices y planes de ejecución

Problema. Cuentas por cobrar revisa los pedidos que vencen en un solo día (15 de julio de 2013) y la consulta recorre toda la tabla de pedidos.

```
SELECT SalesOrderID, CustomerID, SubTotal, TotalDue
FROM Sales.SalesOrderHeader
WHERE DueDate >= '20130715' AND DueDate < '20130716';
```

Requerimiento

1. Ejecutar la consulta con el plan real y `SET STATISTICS IO ON`. Identificar el operador de acceso a la tabla y anotar las lecturas lógicas.
2. Crear el índice (agrupado o no agrupado) que corresponda a la columna de búsqueda `DueDate`. Justifica por escrito tu elección.
3. Repetir la consulta y comparar operador y lecturas lógicas.

Salida esperada. Antes del índice, el plan muestra un Clustered Index Scan (o Table Scan) con lecturas iguales al tamaño de la tabla. Después, el plan debería pasar a un Index Seek sobre el índice nuevo (con Key Lookup si pides columnas que el índice no contiene) y las lecturas lógicas deberían bajar de forma notable. Si el optimizador conserva el Scan, explica por qué con las filas estimadas y las lecturas.

Ejercicio 3 · Optimizando el Key Lookup con INCLUDE

Problema. Tesorería consulta los tipos de cambio publicados en un día concreto (por ejemplo, 1 de julio de 2013) con la moneda destino, la tasa promedio y la tasa de cierre.

```
SELECT CurrencyRateDate, ToCurrencyCode, AverageRate, EndOfDayRate
FROM Sales.CurrencyRate
WHERE CurrencyRateDate = '20130701';
```

Requerimiento

1. Ejecutar la consulta con el plan real y `SET STATISTICS IO ON`. Localizar el Key Lookup y anotar las lecturas lógicas.
2. Crear (o reemplazar el índice existente `AK_CurrencyRate_CurrencyRateDate_FromCurrencyCode_ToCurrencyCode`, documentando tu decisión) un índice no agrupado que cubra la consulta: decide qué columna va como clave y cuáles como columnas incluidas (ToCurrencyCode, AverageRate y EndOfDayRate deben quedar disponibles en el índice).
3. Repetir la consulta y comparar el plan y las lecturas lógicas con las del paso 1. Si conservas ambos índices, verifica en el plan cuál usa el optimizador.

Salida esperada. Al inicio, el plan muestra un Index Seek seguido de un Key Lookup unidos por Nested Loops. Con el índice cubierto desaparece el Key Lookup, queda un único Index Seek y las lecturas lógicas disminuyen. Explica en dos líneas por qué las columnas incluidas no forman parte de la clave de búsqueda.

Ejercicio 4 · Análisis de JOINs y rendimiento (avanzado)

Problema. Tesorería pide, para los territorios de Europa, el número de pedidos de 2013 y su monto en moneda local, por moneda y tipo de tarjeta de crédito.

```
SELECT cr.ToCurrencyCode AS Moneda, cc.CardType AS TipoTarjeta,
       COUNT(*) AS Pedidos, SUM(soh.TotalDue * cr.AverageRate) AS MontoMonedaLocal
FROM Sales.SalesOrderHeader AS soh
JOIN Sales.CurrencyRate AS cr ON cr.CurrencyRateID = soh.CurrencyRateID
JOIN Sales.CreditCard AS cc ON cc.CreditCardID = soh.CreditCardID
JOIN Sales.SalesTerritory AS t ON t.TerritoryID = soh.TerritoryID
WHERE t.[Group] = N'Europe'
  AND soh.OrderDate >= '20130101' AND soh.OrderDate < '20140101'
GROUP BY cr.ToCurrencyCode, cc.CardType
ORDER BY MontoMonedaLocal DESC;
```

Requerimiento

1. Ejecutar la consulta con el plan real, `SET STATISTICS IO ON` y `SET STATISTICS TIME ON`.
2. Documentar las lecturas lógicas de cada tabla (SalesOrderHeader, CurrencyRate, CreditCard y SalesTerritory), el tiempo de CPU y el tiempo transcurrido.
3. Identificar el algoritmo de unión que usó el motor entre cada par de tablas (Nested Loops, Hash Match o Merge Join) y el operador de mayor costo porcentual en el plan.
4. Proponer y crear **un** índice que reduzca el costo de la operación más pesada (indica nombre, columnas clave y columnas incluidas) y volver a medir.
5. Redactar una conclusión de cinco a siete líneas comparando antes y después, y explicando si cambió el algoritmo de unión.

Salida esperada. Capturas del plan y de la pestaña Mensajes antes y después, una lista con lecturas lógicas por tabla, CPU y tiempo transcurrido, el algoritmo de unión de cada par de tablas y el operador más costoso. Tras crear el índice, las lecturas lógicas de la tabla de mayor volumen deberían bajar. Si la mejora es pequeña, justifícalo con el tamaño de las tablas.